



Implementation of the language server protocol and a
Visual Studio Code extension for C/ACSL programming
language features.

August 24, 2024

Internship report

Djamila Mohamed
2023–2024

Supervisor:
Adel Djoudi, Formal Methods Expert

Contents

1	Introduction	1
1.1	Thales DIS	1
1.2	Context	1
1.2.1	Frama-C	1
1.2.2	Problem	1
1.3	Work organization	2
2	Background	2
2.1	The Language Server Protocol	2
2.1.1	Context	2
2.1.2	LSP Requests	3
2.1.3	LSP Capabilities	3
2.1.4	Features	3
2.2	Frama-C plugin development	3
2.2.1	Overview of OCaml	4
2.2.2	Environment Setup	4
2.2.3	Understanding Frama-C's Architecture	4
2.2.4	Plug-in Registration	5
2.2.5	Building with Dune	5
2.2.6	Installing the Plug-in	5
2.2.7	Useful plugins	5
2.2.8		5
2.3	VS Code Extensions	5
2.3.1	TypeScript extension organization	6
2.3.2	Frama-C extensions	6
2.3.3	VS Code IntelliSense	6
3	Design Choices	6
3.1	Language Server architecture	6
3.2	Communication	7
3.3	VS Code subprocess	7
3.4	Handler	8
3.4.1	Command Execution	8
3.4.2	Example Workflow for <code>textDocument/definition</code>	10
3.4.3	Custom Features	10
3.5	A first approach on the server implementation	11
3.6	plug-in installation	11
4	Development	11
4.1	Working example	11
4.2	Project configurations	11
4.3	'Go To Definition' feature	12
4.4	File Synchronization & Diagnostics	12
4.5	File parsing	13
5	LSP use cases with VS Code	13
5.1	Illustration of the implementation	17
6	Future work	17

1 Introduction

1.1 Thales DIS

Thales Digital Identity and Security is the global business unit of Thales which operates in the field of secure software, providing biometric and encryption solutions. Within DIS is the Identity and Biometric Solutions business line which is in charge of digital identity (e.g.: cards and passports).

The IBS BL is divided in three subunits: ID Documents Solutions (IDDS), Digital ID & Verification Solutions and Public Security. I am assigned to the IDDS subunit as an intern software developer, under the supervision of Adel Djoudi, formal methods expert at Thales DIS.

1.2 Context

1.2.1 Frama-C

In IDDS, developers use program analyzers such as Frama-C¹, a C source file analyzer developed by CEA and Inria².

Frama-C is a sophisticated toolkit for analyzing source code written in C. It combines various static and dynamic analysis methods into a single system, allowing different tools included in the suite to build on each other's results for enhanced accuracy.

A key feature of Frama-C is its support for the ANSI/ISO C Specification Language (ACSL), a formal language designed for specifying the behavior of C programs. ACSL facilitates the precise expression of functional properties through function contracts. It enables both human-readable and machine-processable specifications, ensuring a detailed understanding of function behavior.

Frama-C's static analysis is oriented towards ensuring code correctness. Frama-C guarantees the identification of all possible runtime errors and validates adherence to specified functional requirements.

The collaborative framework of Frama-C enhances its analytical power, integrating various tools and allowing them to contribute to a comprehensive examination of code. Its modular and extensible design supports a wide range of analysis techniques, making it a good choice for ensuring code reliability and compliance to detailed functional specifications.

1.2.2 Problem

Frama-C currently lacks a dedicated integrated development environment (IDE) for editing and browsing C code and ACSL annotations, leading to significant inefficiencies in the development process. Writing ACSL specifications often requires manually searching through the Frama-C libraries to find the relevant functions and their definitions. This approach is inefficient.

For those familiar with Frama-C, locating function definitions involves opening a terminal and use the `frama-c -print-share-path` command to identify the directory where Frama-C libraries are stored. Following this, the `grep` utility can be used to search for specific predicate definitions within these files. For example, the command `grep -r 'predicate_name' /path/to/frama-c/libraries` is used to recursively search for occurrences of `predicate_name` within the directory. While this method can yield results, it requires manual navigation and is prone to inefficiencies, especially in large codebases with extensive libraries. Below is an example with the predicate `valid_read_string`.

¹<https://frama-c.com/html/overview.html>

²two prominent research institutions with overlapping goals in scientific advancement and technology innovation

```

home > user > .opam > 4.13.1_fc28 > share > frama-c > share > libc > C __fc_string_axiomatic.h
260 /*@ axiomatic wcsChr {
272
273 /*@ logic Z minimum(Z i, Z j) = i < j ? i : j;
274 /*@ logic Z maximum(Z i, Z j) = i < j ? j : i;
275 @*/
276
277 /*@ predicate valid_string(L)(char *s) =
278 @ 0 <= strlen(s) && \valid(s+(0..strlen(s)));
279 @
280 @ predicate valid_read_string(L)(char *s) =
281 @ 0 <= strlen(s) && \valid_read(s+(0..strlen(s)));
282 @
283 @ predicate valid_read_nstring(L)(char *s, Z n) =
284 @ (\valid_read(s+(0..n-1)) && \initialized(s+(0..n-1)))
285 @ || valid_read_string(L)(s);
286 @
287 @ predicate valid_string_or_null(L)(char *s) =
288 @ s == \null || valid_string(s);
289 @
290
PROBLEMS 257 OUTPUT TERMINAL PORTS 29
● user@c-JBBKWL3:~/git/L1/T0304764/acsl_lsp$ cd
● user@c-JBBKWL3:~$ grep -r valid_read_string `frama-c -print-share-path` | grep predicate
/home/user/.opam/4.13.1_fc28/share/frama-c/share/libc/ __fc_string_axiomatic.h: @ predicate valid_read_string(L)(char *s) =
○ user@c-JBBKWL3:~$

```

Figure 1: Example of searching predicate definition with grep

Searching through potentially large directories and files using grep is time-consuming, especially when dealing with complex libraries with numerous functions and predicates. This method relies heavily on manual effort and familiarity with command-line tools, which can be error-prone and inconvenient.

The inefficiencies of the current method emphasize the necessity for a simpler solution. A Language Server Protocol (LSP) tailored for ACSL code would offer significant improvements by providing features such as real-time code browsing, automatic function and predicate lookup, contextual information, and integration with the editing environment. Such a tool would greatly enhance productivity by automating these tasks and reducing the reliance on cumbersome manual search processes.

Additionally, those unaccustomed to Frama-C and ACSL annotations can benefit from this solution for getting started and have a better understanding of the specification language.

The goal of this project is to develop a software solution to offer language programming features for C/ACSL in IDEs including code navigation, completion suggestions, and interaction with Frama-C code analysis, to enhance validation and facilitate certification at EAL6/EAL7 levels of the Common Criteria.

1.3 Work organization

2 Background

2.1 The Language Server Protocol

2.1.1 Context

Developers working on large scale projects often need a code editing environment with the adequate tooling for the languages they use in their workspace. Multiple solutions such as Eclipse or IntelliJ IDEA exist for Java projects, as well as other popular programming languages. But the complexity of developing one tool for one language and one specific editor is tedious as it forces developers to implement the same language programming features for every code editor with each programming language. To overcome these, a solution was developed: the Language Server Protocol (LSP)³.

The Language Server Protocol (LSP) operates through a structured communication system between the client (such as an IDE or code editor) and the server (which provides language-specific features). This communication is facilitated by various types of messages and methods that define the interactions between the client and server.

The language Server Protocol (LSP) is built on top of the [JSON-RPC protocol](https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/), a remote procedure call protocol that uses JSON as data format. It permits a communication between a client and a server.

³<https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/>

Figure 2 illustrates an example of usage of the LSP, it demonstrates the data exchanges between the development tool and the language server.

2.1.2 LSP Requests

Request Message: A request message is sent by the client to ask the server to perform a specific action. Each request is associated with a particular method, which determines the type of operation being requested. For example, methods like `textDocument/completion` request code completion suggestions, while `textDocument/definition` asks for the location of a symbol's definition. These requests expect a response from the server, which will either provide the requested data or indicate an error.

Notification Message: Notification messages are used by the client to inform the server of events or changes in the editor, such as opening, modifying, or closing a document. Unlike request messages, notifications do not expect a response. They keep the server updated on the current state of the workspace. Methods associated with notifications include `textDocument/didOpen`, `textDocument/didChange`, and `textDocument/didClose`.

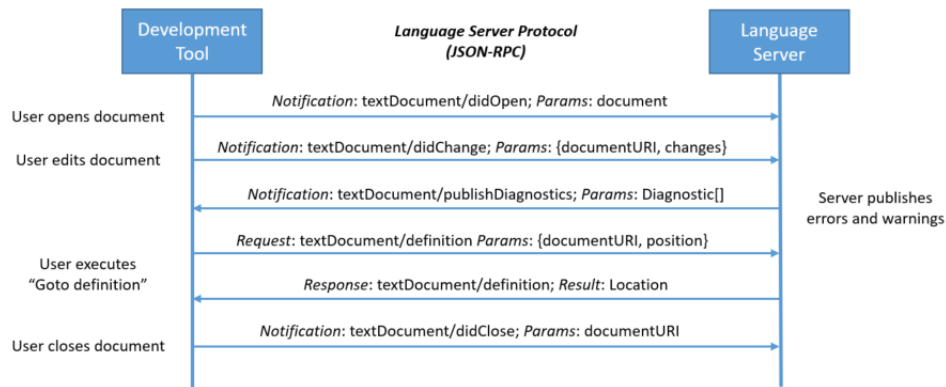


Figure 2: Language Server Protocol sequence diagram: example of an interaction flow between client and server

2.1.3 LSP Capabilities

The LSP defines sets of features named 'capabilities'. LSP capabilities are boolean values transported in a JSON request that each party declares when the extension is launched. These declarations are needed so that both agree on which features each can provide. Only the features supported by both parties can be used in the editor.

Capabilities can be registered in two ways: statically and dynamically. Static registration is done through the initialize request, so it is done once and for all. Dynamic registration allows language servers to add or remove support for various capabilities at runtime without a reinitialization of the server.

Figure 4 shows a sequence diagram of the first requests exchanged between the client and the server, the user can start using the extension once the server has received the 'initialized' notification by the client (not to be confused with the 'initialize' request)

2.1.4 Features

LSP features are organized into categories based on the type of operations they perform. These categories include:

Text Document Features: These features deal with operations on text documents, such as opening, editing, saving, and closing files. Examples include `textDocument/didOpen`, `textDocument/didChange`, and `textDocument/didSave`.

Language Features: These features provide language-specific features such as autocompletion, go-to-definition, hover information, and find-references. Examples include `textDocument/completion`, `textDocument/definition`, and `textDocument/hover`.

Workspace Features: These features relate to operations that affect the entire workspace, such as searching for symbols or managing project settings. Examples include `workspace/symbol` and `workspace/configuration`.

Here is a listing of all LSP features and the use cases associated with it.

2.2 Frama-C plugin development

This document provides a brief overview of the key aspects of developing a Frama-C plug-in, including OCaml's module system and the process of registration, building, and installing a plug-in using Dune.

Table 1: Option details

Client option	Advanced	Definition	Declaration	Display CIL	Callgraph	Local Metrics	Global Metrics	Proof Obligations	Prove Annotation	Diagnostics
acslLsp		X	X	X	X	X	X	X	X	X
kernel.sourceFiles		X	X				X			X
kernel.includePaths		X	X	X	X	X	X	X	X	X
kernel.macros		X	X	X	X	X	X	X	X	X
kernel.machdep				X	X	X	X	X	X	X
kernel.keepUnusedSpecifiedFunctions		X	X	X	X	X	X	X	X	
kernel.aggressiveMerging		X	X	X	X	X	X	X	X	
kernel.generatedSpecCustom	Yes			X	X	X	X	X		
metrics.output					X	X	X			
callgraph.output					X					
callgraph.roots					X					
callgraph.services					X					
wp.rte									X	
wp.checkMemoryModel	Yes								X	
wp.volatile	Yes								X	
wp.pruning	Yes								X	
wp.prover									X	
wp.timeout									X	
wp.session									X	
diagnostics.wp										X

2.2.1 Overview of OCaml

OCaml is a functional programming language with a rich type system. It organizes code into modules, which can be used to encapsulate and manage related functionalities. Key features of OCaml's module system include:

- Modules: Define and group related types, functions, and values. Modules can be implemented in `.ml` files and their interfaces in `.mli` files.
- Functors: Parameterized modules that can create new modules by applying parameters.
- Interfaces: Define the public signature of a module, specifying which functions and types are exposed.

OCaml's type system helps ensure the correctness of complex static analysis tasks by catching errors at compile time. The language's functional programming features, comprising pattern matching, simplify the manipulation of abstract syntax trees and other data structures crucial for program analysis. Additionally, OCaml's modular system facilitates the organization and extension of Frama-C's various analysis components.

2.2.2 Environment Setup

- Frama-C Installation: Ensure Frama-C is installed. It is recommended to use the latest version.
- OCaml Toolchain: Install the OCaml toolchain, including tools like `ocamlfind`, `ocamlbuild`, and `opam`.
- Editor Configuration: Set up a code editor with OCaml support for syntax highlighting and code completion.

2.2.3 Understanding Frama-C's Architecture

- Frama-C Kernel: Provides essential services such as AST (Abstract Syntax Tree) management, project handling, and results storage.
- Existing Plug-ins: Review existing Frama-C plug-ins (e.g., Eva, WP) for structural insights.
- API Documentation: Familiarize with the Frama-C API for functions and modules.

2.2.4 Plug-in Registration

To register a Frama-C plug-in, a file named `plugin.ml` is created. This file includes the plug-in's meta-information and registration code. The typical registration involves defining a module with plug-in details and using `Plugin.register` to integrate the plug-in with the Frama-C system.

2.2.5 Building with Dune

Dune is a build system for OCaml projects that simplifies the build process. To build a Frama-C plug-in using Dune:

- Create a `dune-project` file to define the project settings:

```
(lang dune 2.8)
(name myplugin)
```

- Write a `dune` file to specify build rules, including dependencies and build options:

```
(executable
  (name plugin)
  (libraries frama-c)
  (preprocess (pps ppx_deriving.std))
  (flags (:standard -w +a)))
```

- Build the plug-in with `dune build`, which compiles the plug-in and manages dependencies.

2.2.6 Installing the Plug-in

After building, the plug-in is installed using the command: `dune install`. This command places the plug-in in a directory where Frama-C can find and use it. Verification of the installation involves checking that the plug-in is correctly loaded by Frama-C.

2.2.7 Useful plugins

The following plug-ins are the ones we will be focusing on in our plug-in development process:

Control Flow Graph (CFG)

The CFG plug-in is central to performing control flow analysis in Frama-C. It allows the construction and manipulation of control flow graphs, which represent all possible paths of execution in a program. Plugins that need to analyze or transform the control flow of a program rely heavily on this module.

Deductive Verification (WP)

The WP plugin is used to generate verification conditions using the weakest precondition calculus. These conditions are then checked to ensure that the program adheres to its formal specification, typically expressed in ACSL (ANSI/ISO C Specification Language). This module is crucial for plugins aimed at formal verification or proof-based analysis.

Metrics

This module computes various code metrics, such as cyclomatic complexity and lines of code. These metrics are often used to assess the quality and maintainability of the code, making this module useful for plugins focused on code quality analysis.

2.2.8

2.3 VS Code Extensions

TODO : working example (extension plaintext ?), comment est organisé une extension typescript

2.3.1 TypeScript extension organization

The TypeScript extension is organized with various configuration files and directories crucial for its functionality in Visual Studio Code. Key components include `language-configuration.json` for defining language features, `tsconfig.json` for TypeScript compiler settings, and `package.json` for extension configuration. The `src` directory contains the TypeScript source code that launches the language client, while directories like `node_modules` and `out` manage dependencies and compiled outputs.

The architecture of VS Code extensions is based on a client-server model, which aligns perfectly with the Language Server Protocol. VS Code provides an API with TypeScript libraries for developing different types of extensions, particularly, language extensions. A language extension offers declarative (e.g. syntax highlighting, bracket autoclosing) and programmatic (e.g. hover information, jump to definition) language features.

2.3.2 Frama-C extensions

Very few VS Code extensions for Frama-C exist in the marketplace⁴. The only extensions related to Frama-C are for ACSL syntax coloration made by an independent developer⁵.

The first one, ‘ACSL-lite’⁶ removes syntax coloration for C code and adds coloration for ACSL annotations, but it has no other feature. The drawback of this extension is that it removes entirely the coloration for C when we also need it.

The second one ‘ACSL: ANSI/ISO C Specification Language’⁷ has coloration for both ACSL and C. But there are a few fixes to make.

2.3.3 VS Code IntelliSense

3 Design Choices

3.1 Language Server architecture

The server was implemented as a Frama-C plug-in to fully exploit the advanced parsing and analysis capabilities inherent in the Frama-C framework. By developing the server as a plug-in, we can directly leverage these pre-existing tools.

The plug-in is separated in two modes: the command line mode and the handler mode. The command line mode allows a non-LSP usage of the plug-in, meaning the plug-in will not need to connect to an editor in order to function, in this case, the result will be returned in the terminal in the form of a JSON string, the user will have to be careful of the options they are combining as some of them will produce an unwanted result or an error. The handler mode activates the LSP usage of the plug-in, it will search for a server process listening to the specified port and connect to it, it starts an infinite loop in which a handler manages the incoming and outgoing requests, the commands are decided by the handler and the user’s workspace configurations.

The plug-in consists of four entities:

- **Cmdline_parameters**: Registers plug-in, defines the plug-in’s command line options and the actions associated with them according to their values.
- **Handler**: Contains the code that connects to the editor, the main loop that handles requests, and stores the user’s workspace configurations (sent by the editor).
- **Lib**: OCaml modules that hold the LSP types.
- **Features**: Main features, functions interacting with the AST and other plug-ins to provide results.

⁴The Visual Studio marketplace where VS Code extensions can be installed

⁵<https://github.com/MisterZurg>

⁶<https://github.com/MisterZurg/acsl-lite>

⁷<https://github.com/MisterZurg/acsl-ansi-iso-c-specification-language>

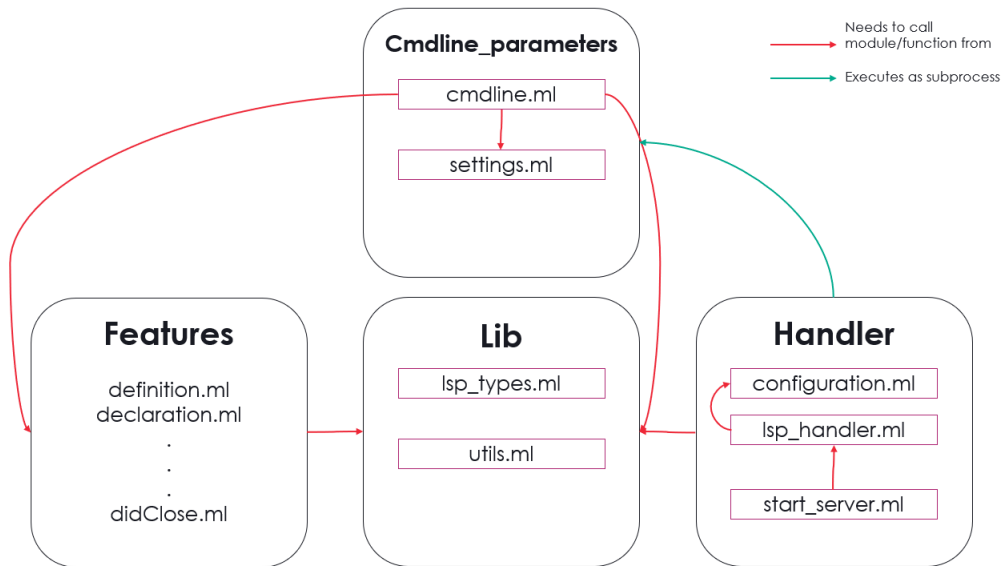


Figure 3: Architecture of the plug-in

3.2 Communication

Unix sockets were selected as the communication mechanism due to their efficiency and simplicity in local inter-process communication (IPC). Unix sockets provide a versatile and reliable means of exchanging data between processes running on the same machine.

Since the processes are all located on the same host, Unix sockets offer a significant performance advantage by avoiding the costs associated with network protocol layers, which are unnecessary in this context.

Furthermore, the OCaml Unix module, which provides native support for Unix sockets, allows ideal integration with the existing OCaml-based infrastructure of the server. This compatibility ensures that data is transmitted efficiently. By leveraging Unix sockets, we achieve a communication mechanism that is not only fast and reliable but also well-suited to the architecture of the application.

3.3 VS Code subprocess

`vscode = serveur`

The language server in this context is not an actual server in the traditional sense, but rather a process that connects to the VS Code server. VS Code manages the communication channels, such as pipes or sockets, by creating them and then allowing the language server process to connect. Essentially, VS Code handles the setup by calling a function starting the server process, and ensuring the connection is established.

Once connected, the two processes can exchange messages. This setup ensures smooth communication, even if the language server is implemented in a different programming language. The Language server is launched via a shell script provided to the client, it runs the following command: `'frama-c -lsp -lsp-debug=4 -lsp-handler'`, the main Frama-C process.

This setup is specific to how VS Code manages extensions. Other editors might handle this connection differently, with varying approaches to managing server processes and communication channels. Only one socket can connect to the VS Code subprocess, meaning that, if another entity were to connect to that port while it is running, its connection will be refused.

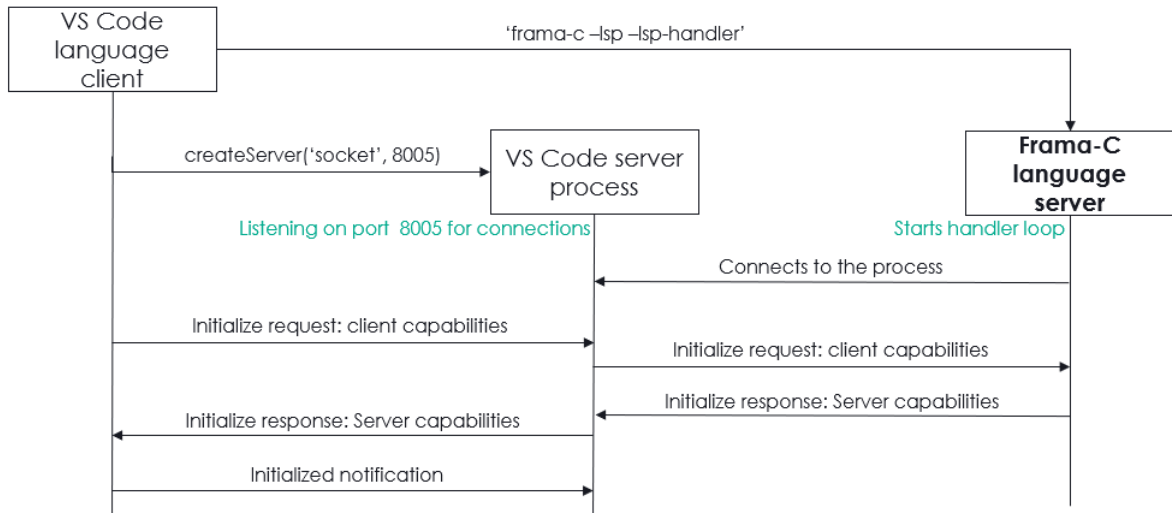


Figure 4: VS Code extension execution and first request exchanges between language client and server

3.4 Handler

The handler is executed in an infinite loop, continuously waiting for incoming requests. It uses the Unix `recv` function, which blocks the process until a request is received. This ensures that the handler remains inactive and consumes minimal resources until an LSP request or notification arrives, at which point it processes the request, sends the response, and then returns to waiting for the next input.

The handler first initializes the server capabilities, which define what the server can do (refer to 2.1.3).

3.4.1 Command Execution

For each LSP request (e.g., `textDocument/definition`, `textDocument/declaration`), the handler constructs a command line to invoke Frama-C with the appropriate options and parameters. These commands are built based on the request's content, such as file paths, line numbers, and character positions and also the current workspace, files being edited, and various global and local parameters (like include paths, macros, other frama-c plug-in options, etc.). The handler then executes the command and processes the output, which it sends back to the client as a response.

If an error occurs during the execution of a command, the handler catches the exception and returns an appropriate LSP error response.

Figure 3 below is a table listing an example of Frama-C command for each feature.

Feature	JSON
Definition	<pre>frama-c test_files/test1.c -cpp-extra-args="-Itest_files/include_folder -Itest_files/include_folder2" -lsp -lsp-debug=0 -lsp-declaration="/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/tes</pre>
Declaration	<pre>{ "jsonrpc": "2.0", "id": 0, "result": [{ "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/ server_v3/test_files/include_folder/test3.h", "range": { "start": { "line": 3, "character": 0 }, "end": { "line": 3, "character": 3 } } }] }</pre>

Display CIL	<pre>{ "jsonrpc": "2.0", "id": 0, "result": "/* Generated by Frama-C */\n/*@ requires z > 7; \n ensures \old(z) > 48; */\nfloat test2(float z)\n{\n float __retres;\n __retres = z + (float)48;\n return __retres;\n}\n\n\n" }</pre>
Callgraph	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { { "type": 3, "message": "Computed <u>callgraph</u> successfully, files generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/test1.dot and /home/ user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/ test_files/test1.c" } } },</pre>
Local Metrics	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/test1.txt" } },</pre>
Global Metrics	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/project_metrics.txt" } }</pre>
Proof Obligations	<pre>{ "jsonrpc": "2.0", "id": 6, "result": "Goal Loop assigns (file test1.c, line 30) (3/3) :\nEffect at line 35\nLet x = to_uint64(1 + i).\nLet a_1 = shift_sint32(a, 0).\nLet a_2 = havoc(Mint_undef_0, Mint_0, a_1, n).\nLet a_3 = shift_sint32(a, i).\nAssume {\n Type: is_uint64(i) /\ \ is_uint64(n).\n (* Heap *)\n Type: (region (a.base) <= 0) /\ \ linked(Malloc_0).\n (* Goal *)\n When: !invalid(Malloc_0, a_3, 1).\n (* Pre-condition *)\n Have: valid_rw(Malloc_0, a_1, n).\n (* Invariant *)\n Have: 0 <= n. \n (* Invariant *)\n Have: (0 <= i) /\ \ (i <= n).\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < i) ->\n (a_2[shift_sint32(a, i_1)] = 0))).\n (* Then *) \n Have: i < n.\n (* Invariant *)\n Have: x <= n.\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < x) ->\n (a_2[a_3 <- 0][shift_sint32(a, i_1)] = 0))).\n} \nProve: included(a_3, 1, a_1, n). \n\n-----\nGoal Loop assigns (file test1.c, line 30) (2/3):\nEffect at line 34\nProve: true. \n\n-----\nGoal Loop assigns (file test1.c, line 30) (1/3):\nProve: true.\n" }</pre>
Prove Annotation	Not implemented
Diagnostics	-metrics, -metrics-output

Table 2: Example Frama-C command for each feature

3.4.2 Example Workflow for textDocument/definition

3.4.3 Custom Features

The handler includes custom commands such as `displayCIL`, `showPOVC`, and metrics-related commands, which are specific to Frama-C. These commands extend the LSP functionality to support domain-specific tasks beyond the standard LSP features.

Figure 3 below is a table listing an example of JSON result for each feature.

Feature	JSON
Definition	<pre>{ "jsonrpc": "2.0", "id": 0, "result": { "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/ server_v3/test_files/test1.c", "range": { "start": { "line": 49, "character": 5 }, "end": { "line": 56, "character": 1 } } } }</pre>
Declaration	<pre>{ "jsonrpc": "2.0", "id": 0, "result": [{ "uri": "/home/user/git/L1/T0304764/acsl_lsp/Acsl_lsp/ server_v3/test_files/include_folder/test3.h", "range": { "start": { "line": 3, "character": 0 }, "end": { "line": 3, "character": 3 } } }] }</pre>
Display CIL	<pre>{ "jsonrpc": "2.0", "id": 0, "result": "/* Generated by Frama-C */\n/*@ requires z > 7; \n ensures \old(z) > 48; */\nfloat test2(float z)\n{\n float __retres;\n __retres = z + (float)48;\n return __retres;\n}\n\n"</pre>
Callgraph	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Computed <u>callgraph</u> successfully, files generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/test1.dot and /home/ user/git/L1/T0304764/acsl_lsp/Acsl_lsp/server_v3/ test_files/test1.c" } },</pre>
Local Metrics	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/test1.txt" } },</pre>
Global Metrics	<pre>{ "jsonrpc": "2.0", "method": "window/showMessage", "params": { "type": 3, "message": "Calculated metrics successfully, file generated : /home/user/git/L1/T0304764/acsl_lsp/ Acsl_lsp/server_v3/test_files/project_metrics.txt" } }</pre>

	<pre> { "jsonrpc": "2.0", "id": 6, "result": "Goal Loop assigns (file test1.c, line 30) (3/3) :\nEffect at line 35\nLet x = to_uint64(1 + i).\nLet a_1 = shift_sint32(a, 0).\nLet a_2 = havoc(Mint_undef_0, Mint_0, a_1, n).\nLet a_3 = shift_sint32(a, i).\nAssume {\n Type: is_uint64(i) /\ is_uint64(n).\n (* Heap *)\n Type: (region (a.base) <= 0) /\ linked(Malloc_0).\n (* Goal *)\n When: !invalid(Malloc_0, a_3, 1).\n (* Pre-condition *)\n Have: valid_rw(Malloc_0, a_1, n).\n (* Invariant *)\n Have: 0 <= n. \n (* Invariant *)\n Have: (0 <= i) /\ (i <= n).\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < i) ->\n (a_2[shift_sint32(a, i_1)] = 0))).\n (* Then *) \n Have: i < n.\n (* Invariant *)\n Have: x <= n.\n (* Invariant *)\n Have: forall i_1 : Z. ((0 <= i_1) -> ((i_1 < x) ->\n (a_2[a_3 <- 0][shift_sint32(a, i_1)] = 0))).\n} \nProve: included(a_3, 1, a_1, n). \n\n-----\nGoal Loop assigns (file test1.c, line 30) (2/3):\nEffect at line 34\nProve: true. \n\n-----\nGoal Loop assigns (file test1.c, line 30) (1/3):\nProve: true.\n" } </pre>
Proof Obligations	
Prove Annotation	Not implemented
Diagnostics	-metrics, -metrics-output

Table 3: Example of corresponding JSON for each feature

TODO : options de notre plugin, celles qui peuvent pas s'activer en meme temps

3.5 A first approach on the server implementation

In the context of Frama-C, an execution instance, whether initiated via the command line or through a function call within the Frama-C library, is referred to as a session. Initially, we considered implementing the server directly as a Frama-C plug-in, where the main loop responsible for handling Language Server Protocol (LSP) requests would be executed within this plug-in. This approach would handle all LSP requests into a single, continuous session.

However, this design presented a significant drawback: when Frama-C encounters a syntax error during file parsing, the entire session enters an error state. Given that a session in this context would persist until the server either crashes or the client terminates it, the error state would persist indefinitely. This is problematic because even if the user corrects the error in the code, the session—and consequently the server—would remain in the error state.

Terminating the session as a response to an error is not a viable solution, as it would lead to the server's shutdown. This, in turn, would cause the client to disconnect, leading to a crash of the extension. Consequently, the user would need to manually restart the extension, which is an undesirable outcome. To avoid such disruptions and maintain a seamless user experience, it is imperative that the server architecture be designed to handle errors more gracefully without necessitating a session termination.

3.6 plug-in installation

4 Development

4.1 Working example

4.2 Project configurations

tableau feature/parsing local (.h ou .c) ou global (spécifié par user ou par default que les fichiers .c)

tableau de chaque feature et sa ligne de commande frama-c (usage de -then et -then-last) faire le lien avec les configs

listing de toutes les options, leur organisation

tableau feature/module api frama-c utilisé tableau algo principal de chaque feature tableau des capacités du serveur

Table 4: Frama-C options available to the user

Client option	Plug-in	Frama-C option(s)
acslLsp	ACSL LSP	-lsp-debug
kernel.includePaths	Kernel	-cpp-extra-args
kernel.macros	Kernel	-cpp-extra-args
kernel.sourceFiles	Kernel	
kernel.machdep	Kernel	-machdep
kernel.keepUnusedSpecifiedFunctions	Kernel	-remove-unused-specified-functions
kernel.aggressiveMerging	Kernel	-aggressive-merging
kernel.generatedSpecCustom	Kernel	-generated-spec-custom
metrics.output	Metrics	-metrics, -metrics-output
callgraph.output	Callgraph	-cg, -cg-roots
callgraph.roots	Callgraph	-cg, -cg-roots
callgraph.services	Callgraph	-cg, -cg-no-services
wp.rte	WP	-wp, -wp-rte
wp.checkMemoryModel	WP	-wp, -wp-check-memory-model
wp.volatile	WP	-wp, -wp-volatile
wp.noPruning	WP	-wp, -wp-no-pruning
wp.prover	WP	-wp, -wp-prover
wp.timeout	WP	-wp, -wp-timeout
wp.session	WP	-wp, -wp-session
diagnostics.wp	ACSL LSP	-lsp-wp

4.3 ‘Go To Definition’ feature

The ‘Go To Definition’ feature of Visual Studio Code allows developers to easily navigate to the definition of symbols within their code. In this context, the challenges involve implementing a VS Code extension that supports this feature for two languages associated with the same file extension (‘.c’ and ‘.h’).

When a user requests to find the definition of a symbol at a specific position in the code, the server first identifies the name of the symbol located at that position. This involves parsing, analyzing the file and extracting the relevant word based on the provided line and column numbers. The code then initiates a search through various subsections of the code to find where the symbol is defined. This search involves visiting different parts of the source file’s Abstract Syntax Tree (AST) and concerns both ACSL and C code:

- ACSL Definitions: The code checks if the symbol matches any logical predicates or terms, which are specific to tools like Frama-C used for static analysis of C programs.
- C Definitions: It examines global entities, such as enums, structures, functions, and variables, to see if the symbol matches any of these.

Each of these searches uses visitor patterns, which traverse and inspect nodes in the AST, comparing each node’s name to the symbol’s name.

If a definition is located, the exact file path and the range of lines and columns where the symbol is defined are packaged into a JSON response. If no definition is found, a response indicating a null result is returned. This response allows the LSP client (the user’s text editor or IDE) to navigate directly to the symbol’s definition or handle the case where no definition exists.

4.4 File Synchronization & Diagnostics

File synchronization is a key feature of the LSP. Three types of requests are sent by VS Code to notify document opening, changing and saving actions performed by the user. A single file parsing is performed each time a file is opened or saved. We add an event listener to catch any error or warning raised by the Frama-C parser and we store them inside a JSON block. All diagnostics are published to the editor and displayed to the user in the form of squiggles and listed in the ‘Problems’ subsection of the VS Code terminal/console (?).

There are two types of diagnostics: warnings and errors. Warnings are raised in case an ACSL annotation is not formed properly or when an unknown function is called. Errors are raised when there is a syntax error in the C code preventing the parser to finish (which are fatal errors). In the latter case, normally, any Frama-C session stops, but we ensure that diagnostics are sent either in case the parsing succeeds or fails. So we send them just before the exception aborts fatally. In case the parsing succeeds without any messages, the diagnostics are cleared in the editor.

Table 5: Features implementation details

Feature	Related Frama-C API modules	Main algorithm	Parsing mode
Go To Declaration	Cil, Visitor, Filepath, File, Ast, Json	Tree traversal of AST	Global
Go To Definition	Cil, Visitor, Filepath, File, Ast, Json	Tree traversal of AST	Global
Diagnostics	Filepath, File, Log, Json	Log event listener	Local
Display CIL of file	Ast, Printer, Pretty_utils, Json	Get computed AST	Local
Show proof obligations (POs) of annotation(s)/file	WP, Property, Filepath, Json, Pretty_utils	List traversal generated POs	Local
Show proof verdict		List traversal	Local
Compute Callgraph	Callgraph	Get computed Callgraph	Local
Show Metrics	Metrics	Get metrics result	Local

The listener is a Frama-C function that reports any event emitted by Frama-C while parsing. From those events we can extract their source, their message and their kind (whether it is an error, a warning a feedback, etc.). By manipulating these data, we organize them and adapt them to the LSP standard.

This feature is only implemented for single file parsing and diagnostics are computed only when the file is saved. No diagnostics are published for multiple file parsing, which means we do not know if those files are in state of error. So the user has to make sure the source files they provided have no errors.

4.5 File parsing

In order to use the Frama-C module’s functions, the plug-in converts the source code files into their C Intermediate Language (CIL) representation, which provides a detailed hierarchical structure of the source code. It captures the syntactic elements and relationships within the code, for deeper analysis and processing. This representation allows the server to efficiently navigate and manipulate the code, supporting features like ‘Go To Definition’ and other code navigation tools.

The system is designed to handle the parsing of both single and multiple source code files. Regardless of the number of files being parsed, the resulting Abstract Syntax Trees (ASTs) from each file are merged into a single one. This merging process is crucial as it simplifies the analysis and manipulation of the code, providing a unified structure that the server can work with more effectively.

When dealing with large codebases, users often face the challenge of working with a vast number of source files. Parsing all these files can be a time-consuming task. To address this issue, the system offers a Visual Studio Code setting that allows users to specify which source files need to be parsed. By focusing the parsing process on only the relevant files, this setting helps optimizing the time spent on parsing, making the workflow more efficient.

Users must ensure that all necessary source files are included in the parsing process. If a user fails to specify a source file that is needed for certain features, such as Go To Definition, they may not function correctly. It is therefore the responsibility of the user to provide all the relevant files.

5 LSP use cases with VS Code

In this enclosed figure is a listing of all LSP features, if they are handled by VS Code, and their implementation status on the server side.

LSP features	Use case(s)	Implementation status
Language features		
Go to Declaration	Show declaration of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented

Go to Definition	Show definition of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented
Go to Type Definition	Show definition of C library, Frama-C and user defined functions types	Implemented (included in go to definition)
Go to Implementation	Show implementation of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	
Find References	Show references of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	
Prepare Call Hierarchy	Prepare call hierarchy of C functions, ACSL terms and predicates	
Call Hierarchy Incoming Calls	List incoming calls of C functions	
Call Hierarchy Outgoing Calls	List outgoing calls of C functions	
Prepare Type Hierarchy	Prepare type hierarchy of C types	
Type Hierarchy Super Types	List super types of C types	
Type Hierarchy Sub Types	List sub types of C types	
Document Highlight	Highlight occurrences of all words OR only C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates	
Document Link	Shows links written inside C and ACSL comments and make them browsable	
Document Link Resolve	Related to Document Link	
Hover	Show C function contract, show C function type information and documentation if available	
Code Lens	Show C function signature	
Code Lens Refresh	Refresh C function signature if changes were made	
Folding Range	Fold function contracts, functions, function descriptions, comments, ghost code	
Selection Range	Suggest selection range of function contracts, functions, function descriptions, comments, ghost code	
Document Symbols	List (in the current file) symbols of C functions, variables, ACSL terms and predicates.	
Semantic Tokens	Request semantic tokens for C and ACSL keywords to provide additional color information.	
Inline Value	Show, at the end of the line, the values of C variables in a line	
Inline Value Refresh	Refresh the values of C variables in a line if changed	
Inlay Hint	No use cases found	
Inlay Hint Resolve	No use cases found	
Inlay Hint Refresh	No use cases found	
Moniker	No uses cases found	
Completion Proposals	Show completion item for ACSL : ACSL keywords completion (ensures, assigns, etc.), propose variables bound to the current file or workspace, C keyword completion.	Ongoing but disabled
Completion Item Resolve	Extend Completion Proposals feature	
Publish Diagnostics	Show compilation errors, warnings for C and ACSL code.	
Pull Diagnostics	Related to Publish Diagnostics	
Signature Help	Show C library, Frama-C and user defined function signatures when function name is typed	
Code Action	The server can provide code actions related to a diagnostic such as fixing includes, etc.	
Code Action Resolve	Related to Code Action	
Document Color	Show color of ANSI color codes in the C code if any (when printing colors in the terminal, etc.)	
Color Presentation	Related to Document Color	

Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators.	
Range Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators for selected lines in the code.	
On type Formatting	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semi-colon, spaces between symbols, arithmetic and logic operators as user types according to formatting trigger characters provided by the server. Trigger characters being ";" and "}" so that formatting is done after writing a line or a block.	
Rename	Workspace-wide rename of a symbol (C functions, variables, macros, types, ACSL types, ACSL terms and predicates)	
Prepare Rename	Related to Rename	
Linked Editing Range	No use cases found yet	
Workspace features		
Workspace Symbols	Helps users find and list symbols across a workspace based on search queries	
Workspace Symbol Resolve	Related to Workspace Symbols	
Get Configuration	Get workspace settings: include paths, source files, macros	Implemented
Did Change Configuration	Get configurations each time user changes them	Implemented
Workspace Folders	Allow support of multiple roots folders	
Did Change Workspace Folders	Be notified each time a workspace configuration is modified	Implemented
Will Create Files	Get workspace edits which will be applied to workspace before the files are created	
Did Create Files	Get notified when files are created from within the client	
Will Rename Files	Get notified of file renames before files are actually renamed as long as the rename is triggered from within the client either by a user action or by applying a workspace edit	
Did Rename Files	Get notified when files are renamed from within the client	
Will Delete Files	Get notified of file deletes before deleting them	
Did Delete Files	Get notified when file are deleted	
Did Change Watched Files	Get notified when files outside the primary editing scope, such as makefiles or python/bash scripts related to the project	
Execute Command	Generate boilerplate code, a function contract skeleton	
Apply Edit	Automatically insert or update ACSL annotations in the code after a refactoring operation, ensuring that the formal specifications remain coherent with the modified code	
Window features		
Show Message Notification	Notify the user when a file is generated by one of the plug-ins (callgraph, metrics, ...)	Implemented for callgraph and m
Show Message Request	If the user asked for the proof obligation of a precondition of a function, asks for which function call it should show the proof obligations)	
Log Message	No use case found	
Show Document	Show generated callgraph	

Create Work Done Progress	Display proof verdict calculation status bar	
Cancel a Work Done Progress	Related to work done progress	
Sent Telemetry	Enable the collection of usage and performance data to monitor server performance, analyze feature usage, track errors, and help development improvements	

Table 6: LSP features use cases

tableau des capacités de vs code

VS Code features	Use cases	Implementation status
Programmatic		
Provide Diagnostics	Show compilation errors, warnings for C and ACSL code.	Implemented
Show Code Completion Proposals	ACSL keywords completion (ensures, assigns, etc.), propose variables bound to the current file or workspace, C keyword completion.	VS Code supports word based completions for any programming language
Show Hovers	Show C function contract, show C function type information and documentation if available	
Help With Function and Method Signatures	Show C library, Frama-C and user defined function signatures when function name is typed	
Show Definitions of a Symbol	Show definition of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	Implemented
Find All References to a Symbol	Show references of C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates.	
Highlight All Occurrences of a Symbol in a Document	Highlight occurrences of all words OR only C library, Frama-C and user defined functions, variables, macros, types, ACSL types, ACSL terms and predicates	
Show all Symbol Definitions Within a Document	Show all symbols in the document. Define the kinds of symbols such as variables, functions	
Show all Symbol Definitions in Folder	Show all symbols define by the source code within the open folder. Define the kinds of symbols such as variables, functions	
Possible Actions on Errors or Warnings	Fix missing semi-colons for C and ACSL code, unclosed brackets and parenthesis	
CodeLens - Show Actionable Context Information Within Source Code	Show C function signature	
Show Color Decorators	Show color of ANSI color codes in the C code if any (when printing colors in the terminal, etc.)	

Format Source Code in an Editor	Fix indentations in C file for C and ACSL code. Provide indentation levels for ACSL code. Carriage return after each semicolon, spaces between symbols, arithmetic and logic operators.	
Format the Selected Lines in an Editor	Same as above but for selected lines in the code.	
Incrementally Format Code as the User Types	Same as above but code formats automatically as user types according to formatting trigger characters provided by the server. Trigger characters being ";" and "}" so that formatting is done after writing a line or a block.	
Rename Symbols	Rename all occurrences of a symbol, it should differentiate between functions, predicates, terms, variables and constants as they can have the same name.	
Declarative		
Syntax highlighting		Implemented
Snippet completion		Implemented
Bracket matching		Implemented
Bracket autoclosing		Implemented
Bracket autosurrounding		Implemented
Comment toggling		
Auto indentation		
Folding		

Table 7: VS Code Programmatic and Declarative Features

5.1 Illustration of the implementation

commenter les screenshots de chaque feature

6 Future work

tableau des features à implémenter